Assignment 5 Detailed Report Markov Decision Process (MDP) Analysis

1. Introduction

This assignment focuses on solving a Markov Decision Process (MDP), which is a mathematical framework used for modeling decision-making in situations where outcomes are partly random and partly under the control of a decision-maker.

The problem models a robot that operates with different battery levels. The robot must decide whether to continue working (Search) or conserve energy (Wait) in order to maximize long-term rewards.

We apply three major reinforcement learning techniques: Policy Evaluation, Value Iteration, and Policy Iteration to analyze and solve the problem.

2. MDP Formulation

States: High, Low, Charging

Actions: Search, Wait

Discount Factor (γ): 0.9

Convergence Threshold (θ): 1e-6

The transition probabilities define how the robot moves between states depending on its action. Rewards are associated with each action-state pair.

Important Modeling Note: The transition table provides different rewards for the same action (Low→Search) depending on the next state. Since the assignment requires a 2D reward matrix R[s, a], we compute the expected reward as:

R(Low, Search) = 0.4*(-3) + 0.6*(4) = 1.2

3. Task 1: Iterative Policy Evaluation

Policy Evaluation computes the value function V(s) for a fixed policy using the Bellman expectation equation.

The given policy is:
High → Search
Low → Wait
Charging → Wait

We iteratively update values until convergence.

Final Values:
High ≈ 18.11
Low ≈ 10.00
Charging ≈ 16.30

Interpretation: High has the highest value due to frequent rewards. Low has lower value due to limited reward accumulation. Charging inherits value from High because it transitions directly.

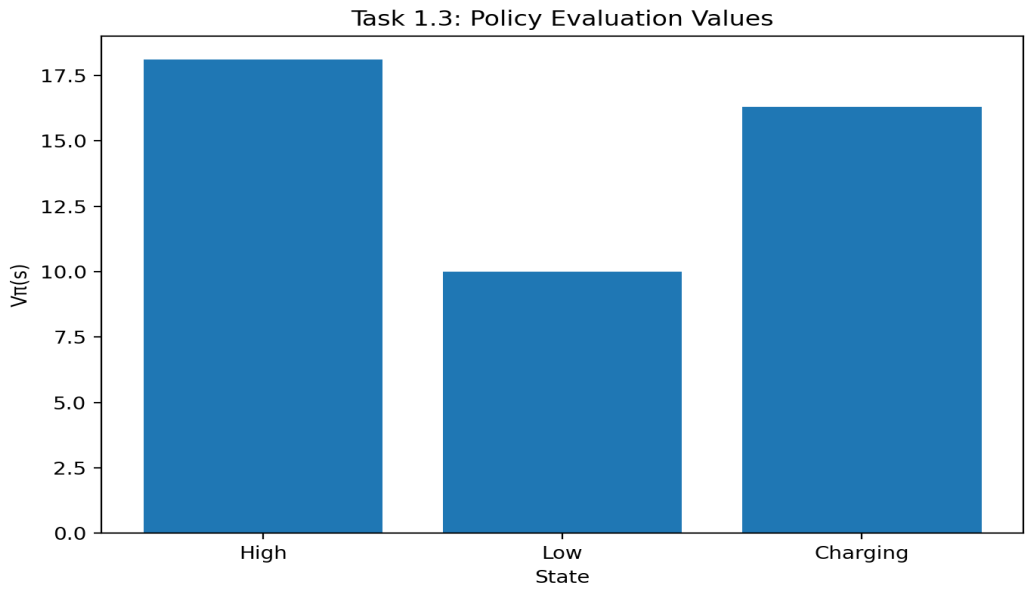

Figure 1: Value of each state under given policy

4. Task 2: Value Iteration

Value Iteration computes the optimal value function directly using the Bellman optimality equation. At each step, the best possible action is chosen.

This process continues until values converge within the threshold.

Convergence: 142 iterations

Optimal Policy:
High → Search
Low → Search
Charging → Wait

Optimal Values:
High ≈ 29.64
Low ≈ 25.81
Charging ≈ 26.68

Insight: Even in Low state, Search is preferred because its expected reward is higher than Wait and it provides a chance to return to High.

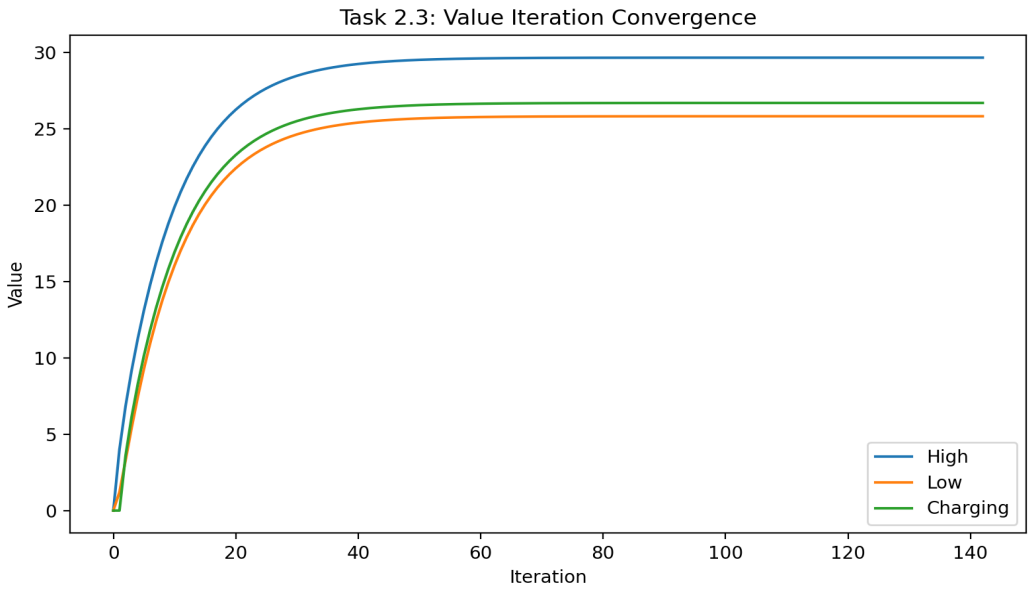

Figure 2: Convergence of Value Iteration

5. Task 3: Policy Iteration

Policy Iteration alternates between policy evaluation and policy improvement.

Starting with an initial policy (all Wait), the algorithm evaluates the policy and improves it greedily.

Iterations: Converged in 2 steps

Final Policy:
High $\rightarrow$ Search
Low $\rightarrow$ Search
Charging $\rightarrow$ Wait

Observation: Policy Iteration converges faster than Value Iteration because it updates policies directly rather than gradually improving values.

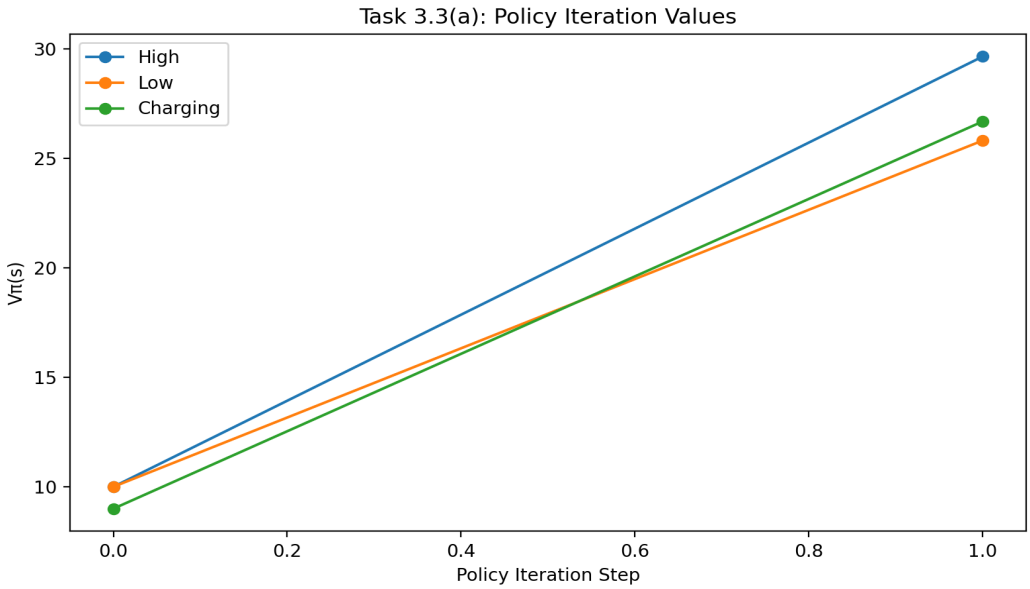

Figure 3: Value evolution during Policy Iteration

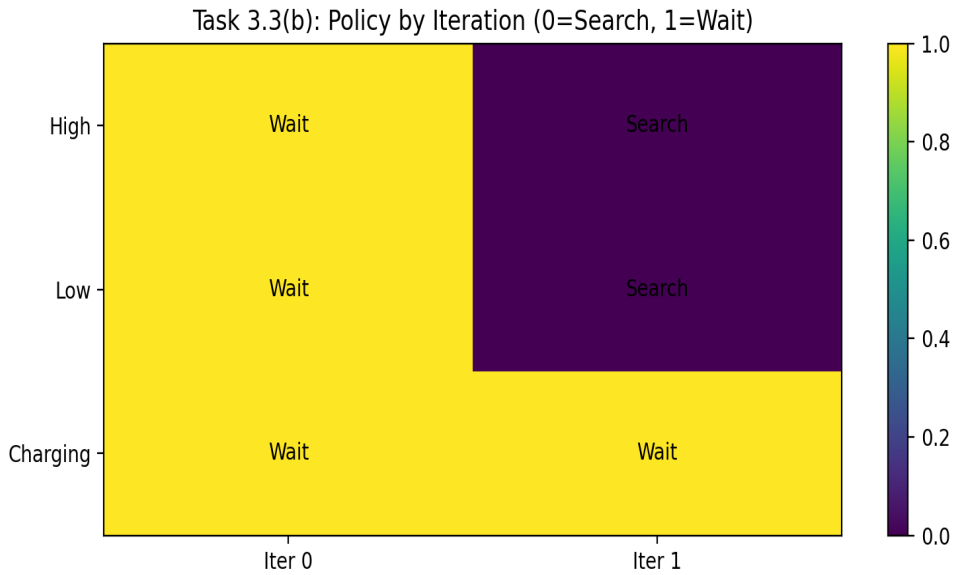

Figure 4: Policy evolution across iterations

6. Comparative Analysis

Value Iteration vs Policy Iteration:

Value Iteration required many iterations to converge but updates values directly. Policy Iteration converged faster because it makes stronger updates by improving policies explicitly.

Convergence Behavior:

All values start from zero and increase as iterations progress. Value Iteration uses a max operator at each step, while Policy Evaluation follows a fixed policy.

Optimal Policy Explanation:

Search is optimal in High due to high reward and stability. In Low, Search is still beneficial due to expected reward advantage. Charging has only one valid action (Wait).

Real-World Insight:

This model represents how intelligent systems balance performance and resource constraints. It ensures efficient use of energy while maximizing productivity.

7. Conclusion

All methods converge to the same optimal policy. Policy Iteration is more efficient in terms of convergence speed, while Value Iteration provides a more gradual understanding of value propagation.

This assignment demonstrates the practical importance of dynamic programming techniques in decision-making systems.